

GDB 调试系统调用流程实验报告

1. 实验目标

使用双重 GDB 方案观察系统调用的完整流程：

- 用户态 (U mode) 通过 `ecall` 指令触发系统调用
- 内核态 (S mode) 处理系统调用
- 通过 `sret` 指令返回用户态

2. 实验环境

2.1 环境配置

- QEMU: qemu-4.1.1 (带调试信息版本)
- GDB:
 - x86_64 gdb (调试 QEMU 进程)
 - riscv64-unknown-elf-gdb (调试 ucore 内核)
- OS: ucore Lab5

2.2 修改 Makefile 使用带调试信息的 QEMU

```
# 使用带调试信息的 QEMU 以支持双重 GDB 调试
QEMU := /root/Downloads/qemu-4.1.1/riscv64-sofmmu/qemu-system-riscv64
```

3. 双重 GDB 调试方案

3.1 三个终端的角色

双重 GDB 调试架构

- 终端1: QEMU 模拟器
- └─ `make debug` (启动 QEMU, `-s -S` 参数等待 GDB 连接)
- 终端2: x86_64 GDB (调试 QEMU 本身)
- └─ `attach <QEMU_PID>`
 - └─ 在 QEMU 源码的关键函数设置断点
 - └─ 观察 QEMU 如何处理 `ecall/sret` 指令
- 终端3: riscv64-unknown-elf-gdb (调试 ucore)
- └─ `target remote localhost:1234`
 - └─ 在用户程序 `syscall` 函数设置断点
 - └─ 控制 ucore 执行到 `ecall` 指令

3.2 调试流程

1. 终端1: 启动 QEMU

```
make debug
```

2. 终端2: 附加到 QEMU 进程

```
pgrep -f qemu-system-riscv64 # 获取 PID
sudo gdb
(gdb) attach <PID>
(gdb) handle SIGPIPE nostop noprint
(gdb) break riscv_cpu_do_interrupt
(gdb) break helper_sret
(gdb) continue
```

3. 终端3: 连接 ucore 并加载用户程序符号

```
make gdb
(gdb) add-symbol-file obj/__user_exit.out
(gdb) break user/libs/syscall.c:26
(gdb) continue
```

4. QEMU 处理 ecall 的源码分析

4.1 关键源码文件

文件	作用
target/riscv/insn_trans/trans_privileged.inc.c	ecall/sret 指令的 TCG 翻译
target/riscv/cpu_helper.c	中断处理核心函数
target/riscv/op_helper.c	sret 的 helper 实现
accel/tcg/cpu-exec.c	CPU 执行主循环

4.2 ecall 指令翻译 (trans_ecall)

```
// target/riscv/insn_trans/trans_privileged.inc.c:21
static bool trans_ecall(DisasContext *ctx, arg_ecall *a)
{
    /* always generates U-level ECALL, fixed in do_interrupt handler */
    generate_exception(ctx, RISCV_EXCP_U_ECALL);
    exit_tb(ctx); /* no chaining */
    ctx->base.is_jmp = DISAS_NORETURN;
    return true;
}
```

关键点：

- `generate_exception()` 生成异常，触发中断处理
- `exit_tb()` 退出当前翻译块，不允许链接到其他块
- 所有 `ecall` 初始都标记为 `U_ECALL`，在中断处理时根据当前特权级修正

4.3 中断处理函数 (riscv_cpu_do_interrupt)

```
// target/riscv/cpu_helper.c:503
void riscv_cpu_do_interrupt(CPUState *cs)
{
    RISCVCPU *cpu = RISCV_CPU(cs);
    CPURISCVState *env = &cpu->env;

    // 判断是同步异常还是异步中断
    bool async = !(cs->exception_index & RISCV_EXCP_INT_FLAG);
    target_ulong cause = cs->exception_index & RISCV_EXCP_INT_MASK;

    // ecall 原因映射表
    static const int ecall_cause_map[] = {
        [PRV_U] = RISCV_EXCP_U_ECALL, // 8
        [PRV_S] = RISCV_EXCP_S_ECALL, // 9
        [PRV_H] = RISCV_EXCP_H_ECALL, // 10
        [PRV_M] = RISCV_EXCP_M_ECALL  // 11
    };

    // 根据当前特权级修正 ecall 原因
    if (cause == RISCV_EXCP_U_ECALL) {
        cause = ecall_cause_map[env->priv];
    }

    // 检查是否委托给 S 态处理
    if (env->priv <= PRV_S && ((deleg >> cause) & 1)) {
        // S 态处理
        env->mstatus = ... // 保存状态
        env->scause = cause; // 设置异常原因
        env->sepc = env->pc; // 保存返回地址
        env->pc = env->stvec; // 跳转到中断向量
        riscv_cpu_set_mode(env, PRV_S); // 切换到 S 态
    } else {
        // M 态处理
        ...
    }
}
```

4.4 调试结果 - ecall 处理

```
=== riscv_cpu_do_interrupt 被调用 ===
exception_index = 8 (RISCV_EXCP_U_ECALL)

调用栈:
#0 riscv_cpu_do_interrupt (cs=0x5e69a00d6230)
   at cpu_helper.c:504
```

```
#1  cpu_handle_exception (cpu=0x5e69a00d6230)
    at cpu-exec.c:506
#2  cpu_exec (cpu=0x5e69a00d6230)
    at cpu-exec.c:712
#3  tcg_cpu_exec (cpu=0x5e69a00d6230)
    at cpus.c:1435
#4  qemu_tcg_cpu_thread_fn (arg=0x5e69a00d6230)
    at cpus.c:1743
```

分析：

- `exception_index = 8` 对应 `RISCV_EXCP_U_ECALL` (用户态 `ecall`)
- 调用链: `qemu_tcg_cpu_thread_fn` → `tcg_cpu_exec` → `cpu_exec` → `cpu_handle_exception` → `riscv_cpu_do_interrupt`

5. QEMU 处理 `sret` 的源码分析

5.1 `sret` 指令翻译 (`trans_sret`)

```
// target/riscv/insn_trans/trans_privileged.inc.c:43
static bool trans_sret(DisasContext *ctx, arg_sret *a)
{
    tcg_gen_movi_tl(cpu_pc, ctx->base.pc_next);

    if (has_ext(ctx, RVS)) {
        gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
        exit_tb(ctx);
        ctx->base.is_jmp = DISAS_NORETURN;
    }
    return true;
}
```

5.2 `sret` helper 函数

```
// target/riscv/op_helper.c:74
target_ulong helper_sret(CPURISCVState *env, target_ulong cpu_pc_deb)
{
    // 检查特权级
    if (!(env->priv >= PRV_S)) {
        riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
    }

    // 获取返回地址
    target_ulong retpc = env->sepc;

    // 恢复状态
    target_ulong mstatus = env->mstatus;
    target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);

    // 恢复中断使能
    mstatus = set_field(mstatus, MSTATUS_SIE,
```

```

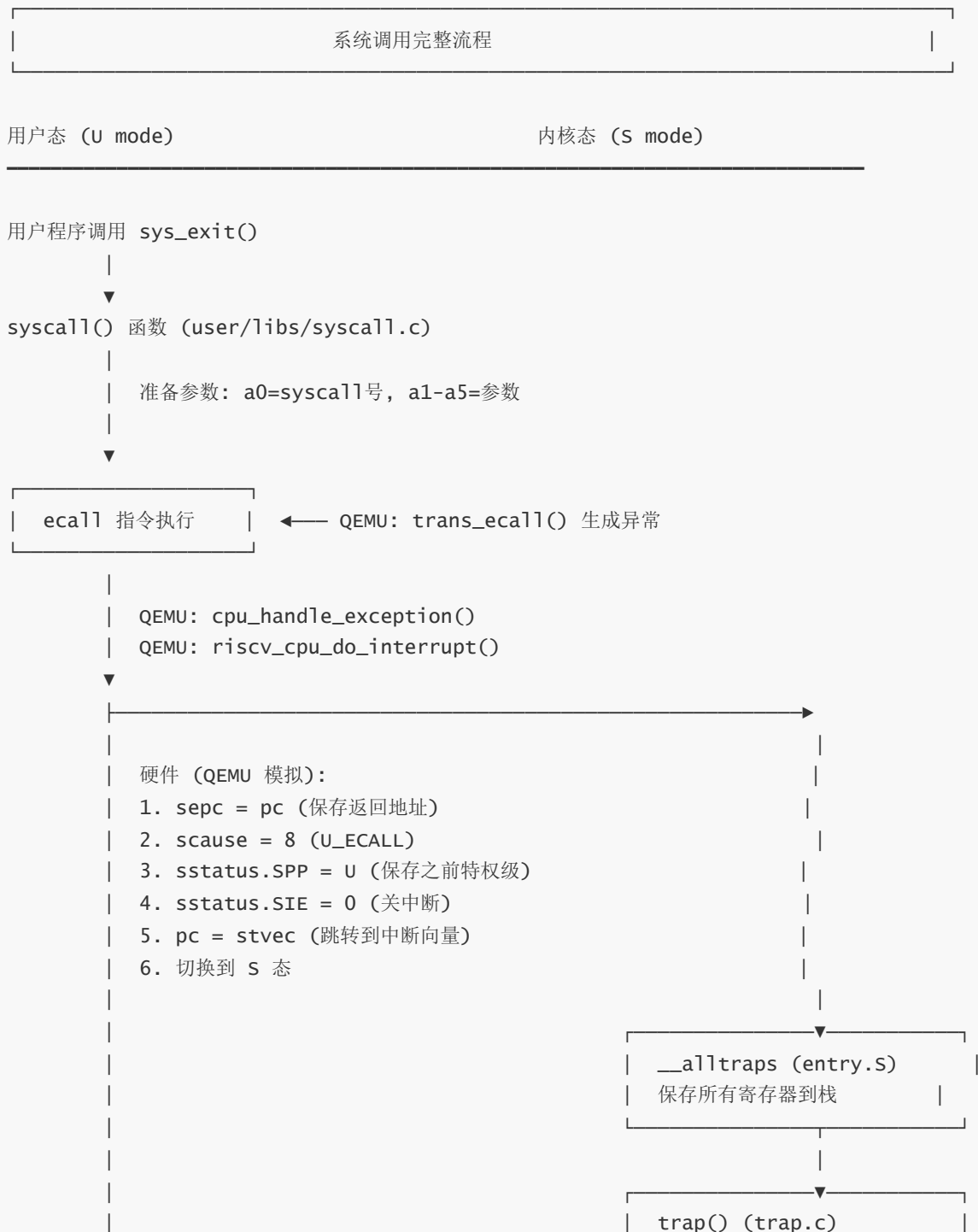
        get_field(mstatus, MSTATUS_SPIE));
mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);

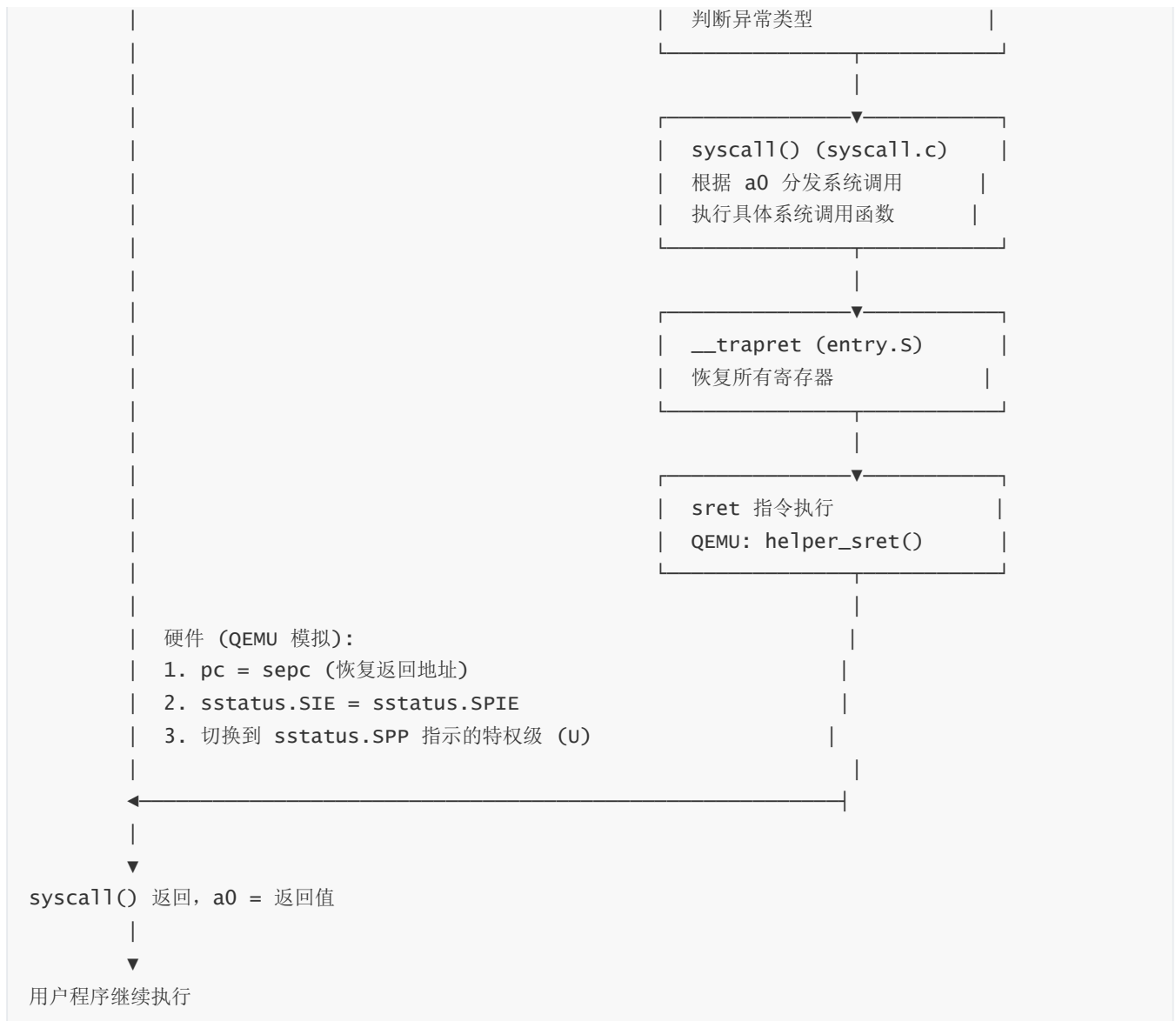
// 切换特权级
riscv_cpu_set_mode(env, prev_priv);
env->mstatus = mstatus;

return retpc; // 返回用户态地址
}

```

6. 系统调用完整流程图





7. TCG 指令翻译机制

7.1 什么是 TCG

TCG (Tiny Code Generator) 是 QEMU 的动态二进制翻译引擎:

- 将客户机指令 (RISC-V) 翻译为宿主主机指令 (x86_64)
- 翻译以"翻译块"(TB) 为单位进行
- 翻译结果被缓存以提高性能

7.2 ecall/sret 的 TCG 处理

对于 ecall 和 sret 这类特权指令:

1. **不直接翻译执行**: 因为涉及状态切换
2. **生成"退出到主循环"的代码**: `exit_tb()`
3. **在主循环中调用 helper 函数**: 如 `helper_sret()`
4. **处理完成后重新进入翻译执行**

TCG 处理 ecall 流程

RISC-V 指令流

QEMU 内部处理

add a0, a1, a2 → TCG 翻译为 x86_64 add 指令
ld a1, 0(sp) → TCG 翻译为 x86_64 mov 指令
ecall → TCG: generate_exception()
 TCG: exit_tb()

```
| 退出 TB, 回到主循环 |  
| cpu_handle_exception() |  
| riscv_cpu_do_interrupt() |  
| 切换特权级, 跳转到 stvec |
```

(stvec 处的代码) ← 继续翻译新的 TB

7.3 与地址翻译调试的关联

在 Lab2 中调试地址翻译时：

- 访存指令被翻译时会插入 **软件 TLB 查找代码**
- TLB 未命中时调用 `get_physical_address()` 进行页表遍历
- 这同样涉及 TCG 翻译和 helper 函数调用

8. 调试过程中的有趣细节

8.1 多次 ecall 的触发

在 ucore 启动过程中观察到多次 `exception_index = 8`，这些来自：

- OpenSBI 初始化时的 SBI 调用
- 内核初始化时的系统调用测试
- 用户程序（如 forktest）的实际系统调用

8.2 SIGPIPE 信号处理

调试 QEMU 时需要 `handle SIGPIPE nostop noprint`，因为：

- QEMU 使用管道进行内部通信
- GDB 默认会在 SIGPIPE 时停止，影响调试

8.3 forktest 成功运行

调试过程中成功观察到 forktest 的完整执行：

```
I am child 31
I am child 30
...
I am child 0
forktest pass.
```

9. 总结

通过本次双重 GDB 调试实验：

1. 理解了 QEMU 如何模拟 `ecall` 指令：

- `trans_ecall()` 生成异常
- `riscv_cpu_do_interrupt()` 处理中断
- 模拟硬件的 CSR 寄存器操作

2. 理解了 QEMU 如何模拟 `sret` 指令：

- `trans_sret()` 调用 helper 函数
- `helper_sret()` 恢复特权级和返回地址

3. 理解了 TCG 翻译机制：

- 普通指令直接翻译执行
- 特权指令通过 helper 函数处理

4. 掌握了双重 GDB 调试技巧：

- 同时调试模拟器和被模拟系统
- 使用条件断点减少干扰